



λFS: Lambda File System

λFS: A Scalable and Elastic Distributed File System Metadata Service using Serverless Functions

Benjamin Carver^{*}, Runzhou Han[‡], Jingyuan Zhang^{*}, Mai Zheng[‡], Yue Cheng[†]

^{*}George Mason University

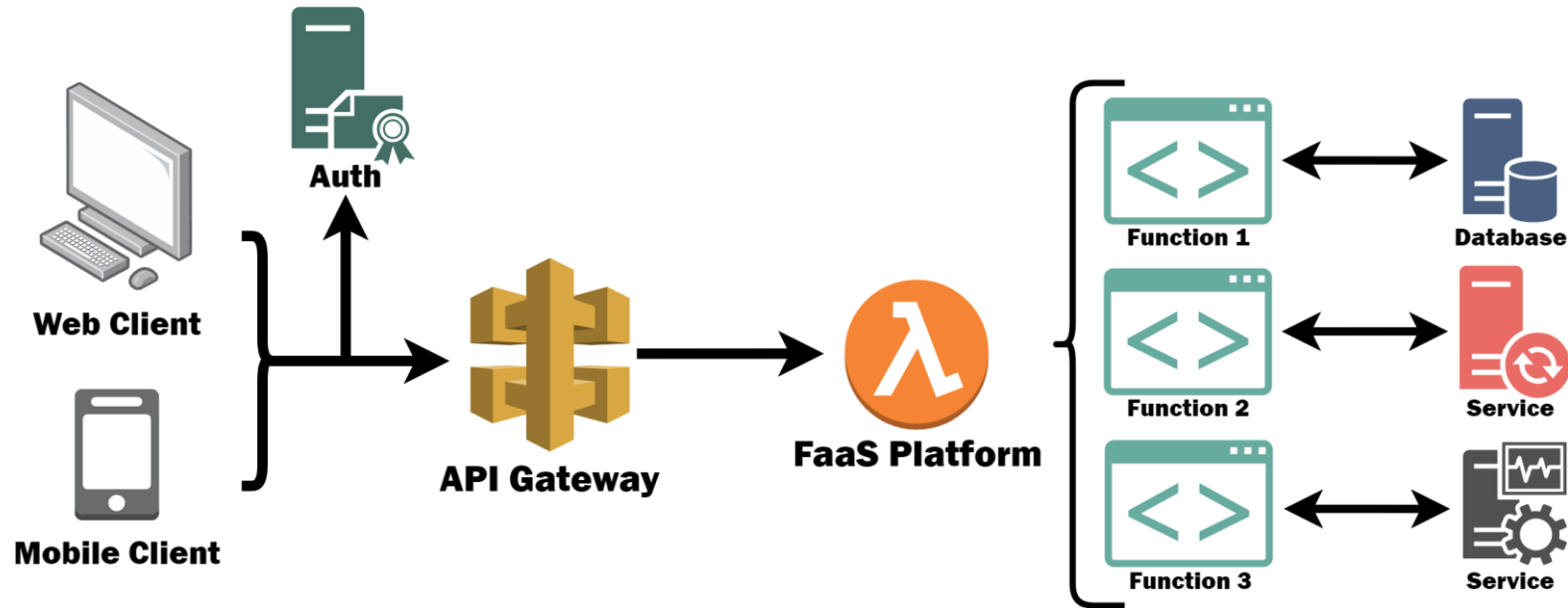
[†]University of Virginia

[‡]Iowa State University



BACKGROUND

Serverless computing (or FaaS) is an emerging **cloud computing** model.



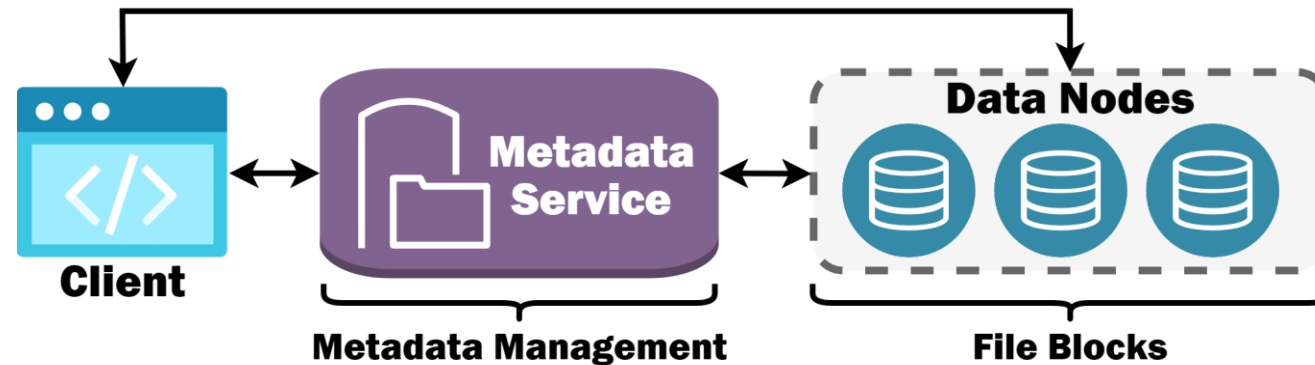
Key benefits of FaaS:

- **Elastic auto-scaling.**
- **Pay-per-use pricing.**
- **Automatic resource management.**



BACKGROUND & MACKGROUND

The **metadata service (MDS)** sits on the **critical path** for **distributed file system (DFS)** operations.



Scaling the **MDS** is **hard**. Existing scaling techniques:

- Lack elasticity.
- Require tedious server management.
- May suffer from high costs & low resource utilization.



MOTIVATION

The question is:

Can we use FaaS in a novel way to build a high-performance, cost-efficient, elastic, and resource-efficient MDS?

Yes! To answer this question, we designed & implemented λ FS.

Serverless functions are an **appealing environment** in which to host the MDS of a large-scale DFS.



**FaaS-based elastic
auto-scaling**



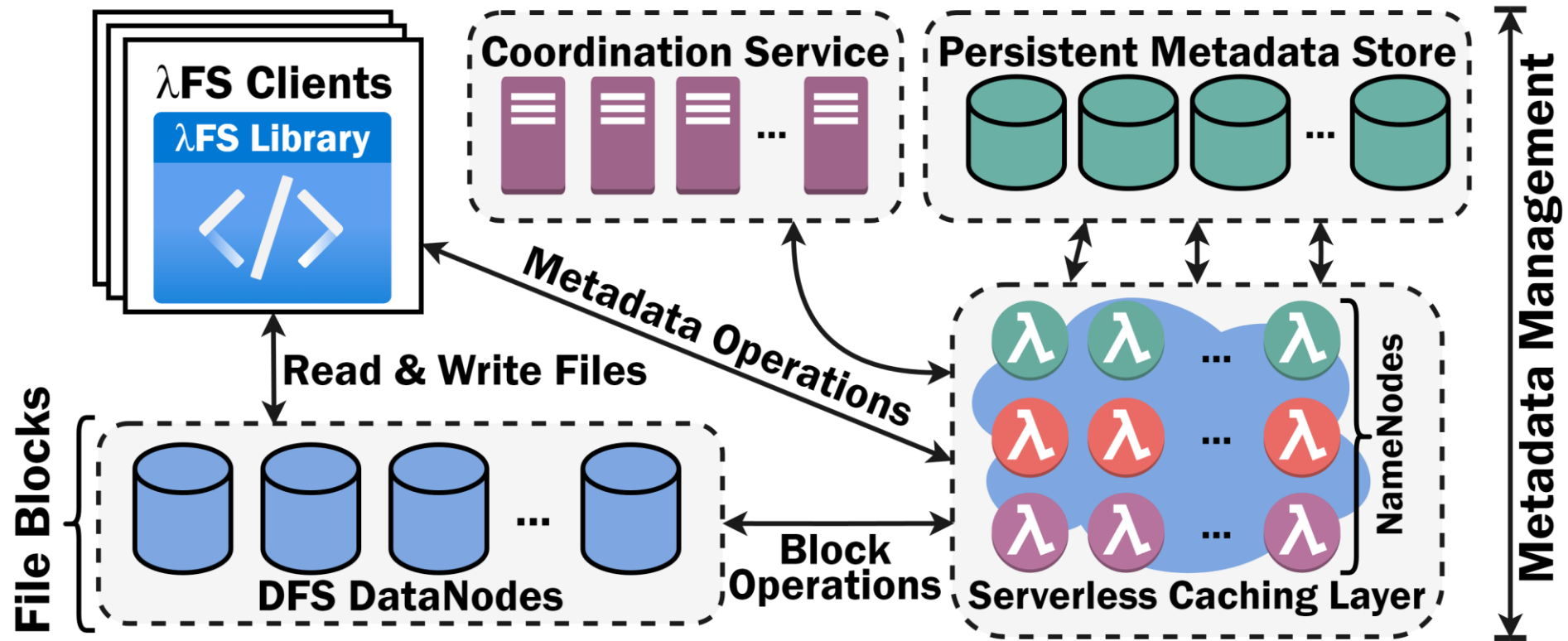
**Pay-per-use
pricing model**



**Automatic resource
provisioning & management**

ARCHITECTURE & DESIGN

λ FS has been ported to both HopsFS (shown below) and BeeGFS+IndexFS.



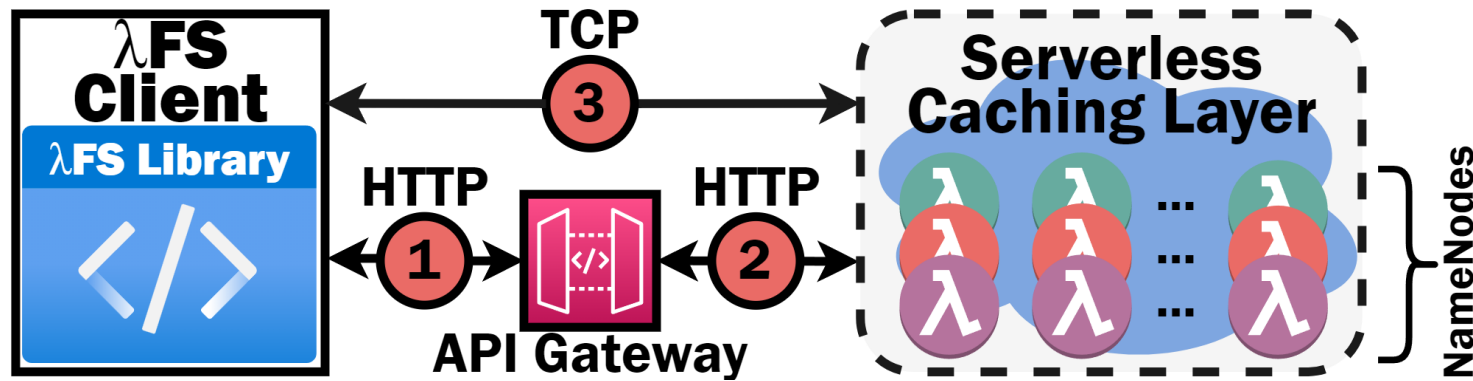
KEY DESIGN COMPONENTS:

- (1) Hybrid, FaaS-Optimized TCP-HTTP RPC Mechanism.
- (2) FaaS-Based Metadata Caching Layer.

DESIGN

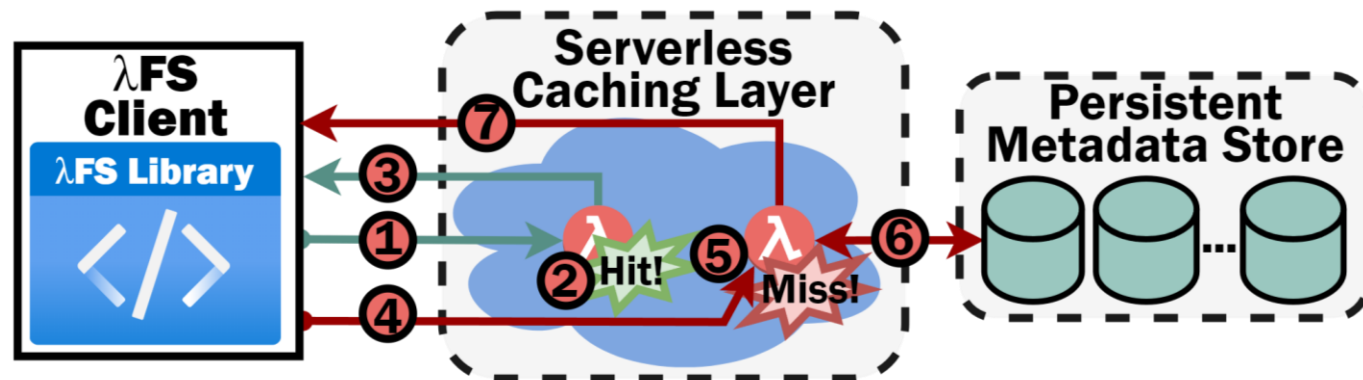
(1) *Hybrid, FaaS-Optimized TCP-HTTP RPC Mechanism*

Clients use a mix of **slow HTTP** and **fast TCP** RPCs to **communicate with serverless NameNodes**.



(2) *FaaS-Based Metadata Caching Layer*

Cache metadata **locally** on NameNodes to **reduce bottleneck** of accessing the persistent backing store.



EVALUATION

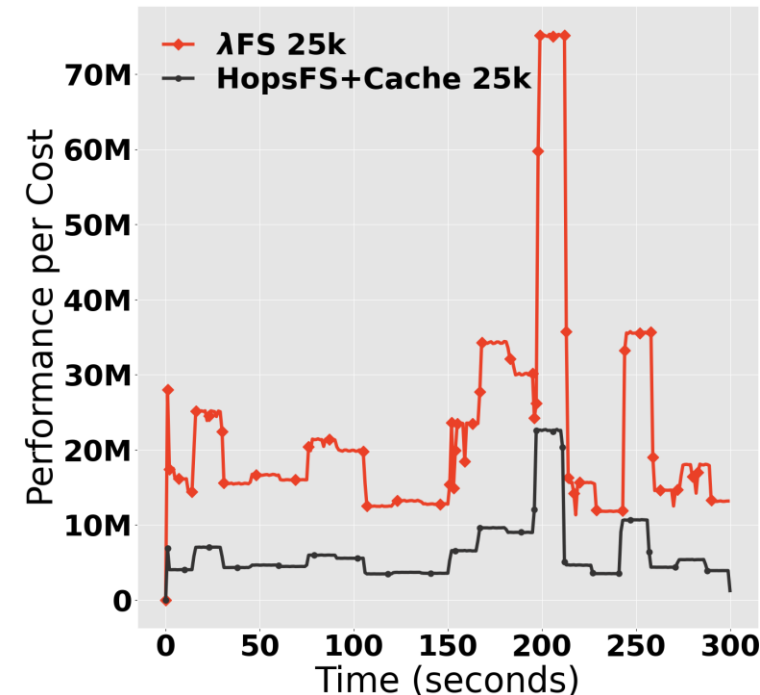
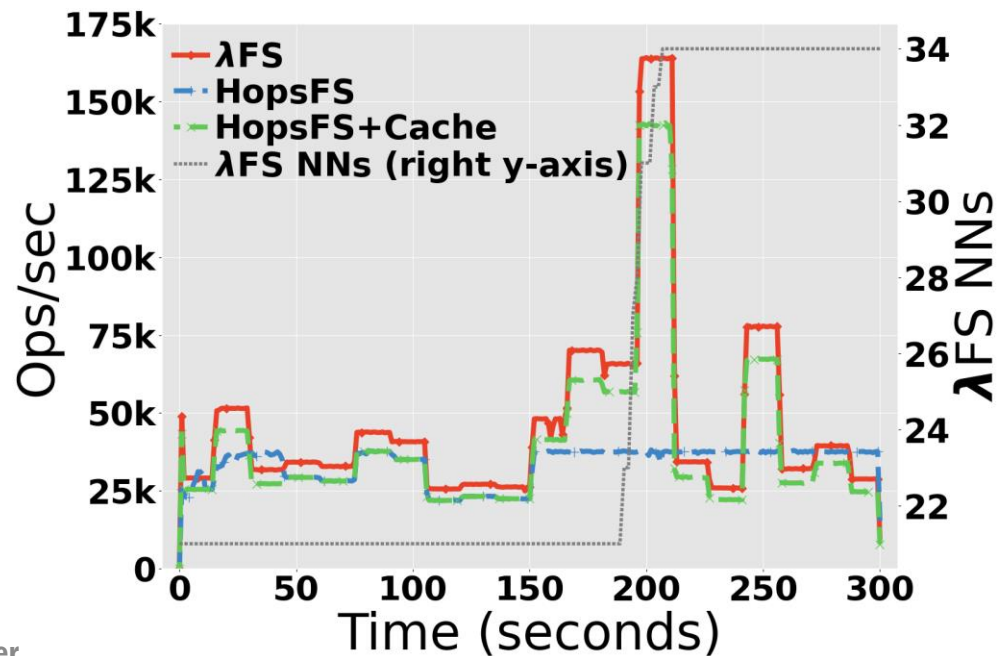
Industrial Workload Performance

- Randomly-generated workload based on **Spotify's HDFS cluster trace**.
- Evaluated auto-scaling policy with **request bursts** (up to **7x base throughput**).
- Evaluated cost-efficiency with *performance-per-cost* metric:

Instantaneous Throughput

Instantaneous Cost

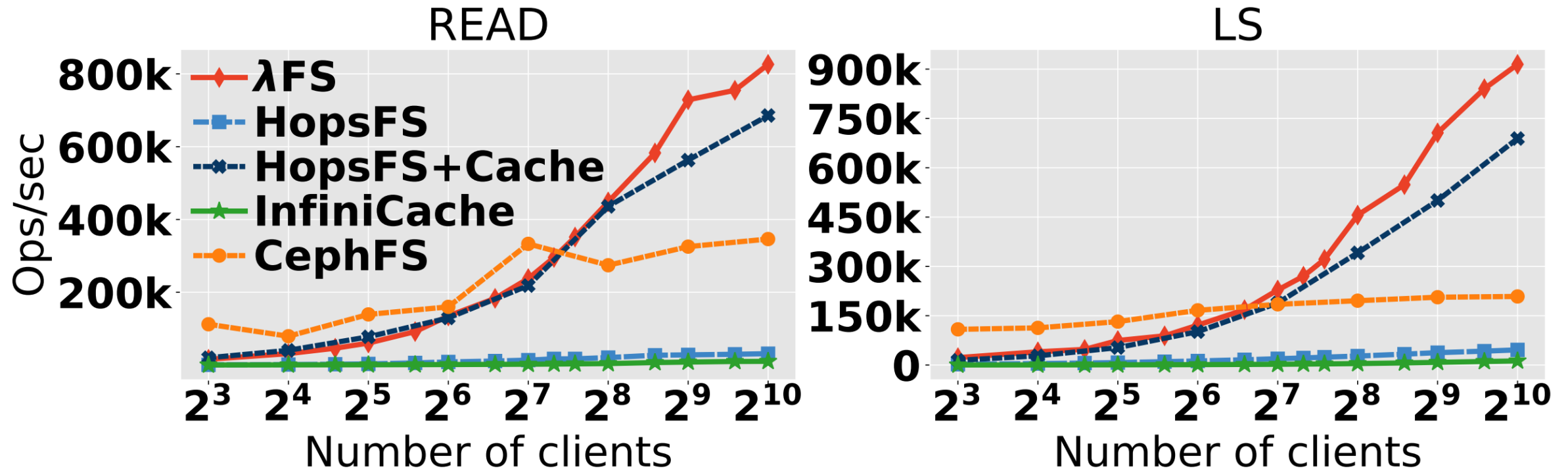
Operation	Percentage	Operation	Percentage
create file	2.7%	read file	69.22%
mkdirs	0.02%	stat file/dir	17%
delete file/dir	0.75%	list file/dir	9.01%
mv file/dir	1.3%	Total Read Ops	95.23%



EVALUATION

Client-Driven Scaling

- 512 total vCPU allocated to each framework; **varied number of clients** from 2^3 (8) to 2^{10} (1,024).
- Each client performed 3,072 FS operations. **Aggregated throughput** plotted on the y-axis.



λ FS ultimately achieved **28.91x** and **20.53x** higher throughput than HopsFS for **read** and **ls**, respectively.

CONCLUSION

λ FS is the **first cloud-native** DFS MDS that uses FaaS to **cache** and **elastically scale** DFS metadata.

λ FS achieves **high-throughput**, **low-latency**, **low cost**, and **high resource efficiency** by utilizing a series of techniques built around a FaaS-based metadata cache.

We have ported λ FS to both HopsFS and IndexFS. We hope that this work will provide insight for building new, cloud-native, performance-sensitive backend services on FaaS.

λ FS will appear at ACM ASPLOS'24 in *λ FS: A Scalable and Elastic Distributed File System Metadata Service using Serverless Functions*. Preprint available: <https://arxiv.org/pdf/2306.11877.pdf>

λ FS Source: <https://github.com/ds2-lab/LambdaFS>

Ds² Research @ UVA: <https://ds2-lab.github.io/>

Benjamin Carver | George Mason University
<https://scusemua.github.io/> | bcarver2@gmu.edu

